OPTIMISED MATRIX MULTIPLICATION APPROACHES AND SPARSE MATRICES

Draft Report Based on the Supplied Experimental Results

Individual Assignment 02 Experimental Report

Rubén Perdomo Aspas

Repositorio: https://github.com/RubenPerdomo159/BigData-Individual-Assigments.git

1. INTRODUCTION.

Matrix multiplication is a useful case study for examining how implementations choices affect on the performance. On on hand, Dense representations store every matrix element, including zeros, in the other hand, Sparse representations aim to reduce work and storage by indexing and converting costs, the results of multiplying sparse matrices can become less sparse.

The experiment have 2 parts where its applied both of this representations. Part A, compares dense implementations, exactly, **baseline**, **reordered**, and **blocked**, while in Part B compares one of these implementations with a CSR-based spare implementation.

2. EXPERIMENTAL SETUP.

The environment supplied was a computer with Windows 11 (AMD64), an Intel processor, 8 logical CPUs, and Python 3.13.12 (64 bits). Both experiments have the same sizes for comparing, 10, 30, 60, 100 and 150, one warm-up run, and five measured repetitions. The CSV recorded the timings in nanoseconds which are converted in miliseconds on the report; the median is used as the summary because is less affected by unusually high or low observations.

The Part A environment file states that the timing covers the multiplication kernel only, its memory field is an estimated matrix-element payload. Part B stores sparse matrices in CSR form using values and pointers for columns and rows, the environment file says that multiplication separates the time from dense-to-CSR, conversión time was measured once per size/sparsity, rather than onec per repetition.

2.1. Data and Correctness Checks.

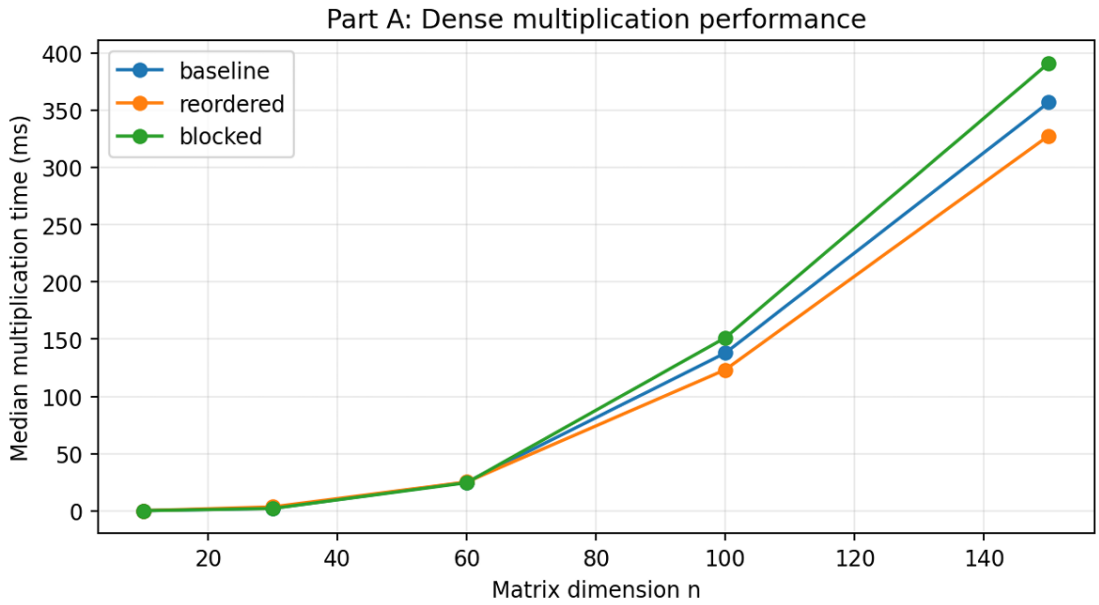
The dense measurements file have 75 rows, including every method, size and repetition. The sparse measurements file have 150 rows, including all methods, sizes, sparsity settings and repetitions. In both files on every rows have *correct = true* which is an evidence of the checks recorded by the benchmarks, it does not replace each inspection of the source code.

On Part B, the sparsity is the target propotion of zero elements, while density is the complementary target of non-zero elements. Since matrices are randomly generated, theirnon-zero counts can differ from the exact target percentages.

3. PART A – DENSE MULTIPLICATION.

The baseline, reordered, and blocked methods were evaluated with the same set of matrix sizes and five repetitions per method and size. The table below reports median multiplication times.

n	Baseline (ms)	Reordered (ms)	Blocked (ms)
10	0.108	0.114	0.119
30	2.238	3.456	2.045
60	25.009	25.223	24.604
100	137.919	123.320	151.284
150	356.803	327.752	391.122



3.1. Observations.

At n=10 the baseline is the fastest (0.108ms) though the differences are very small. At n=30 the blocked method is the fastest (2.045ms), beating the baseline (2.238ms), with reordered being the slowest (3.456ms). At n=60 the three medians are close: blocked is 24.604 ms. Baseline is 25.009 ms. Reordered is 25.223 ms. At n = 100, reordered has the lowest median (123.320ms), baseline (137.919ms) and blocked (151.284ms). At n = 150, reordered is again Fastest (327.752ms), then baseline (356.803ms) and blocked (391.122ms).

The measurements do not support a claim that one optimisation wins at every size. The relative performance changes with matrix dimension. Some repetitions also vary noticeably, particularly for the baseline at n = 100 and reordered at n = 150. For this reason, the medians are more suitable than a single run for comparing the tested methods, and the results should be interpreted as implementation- and environment-specific.

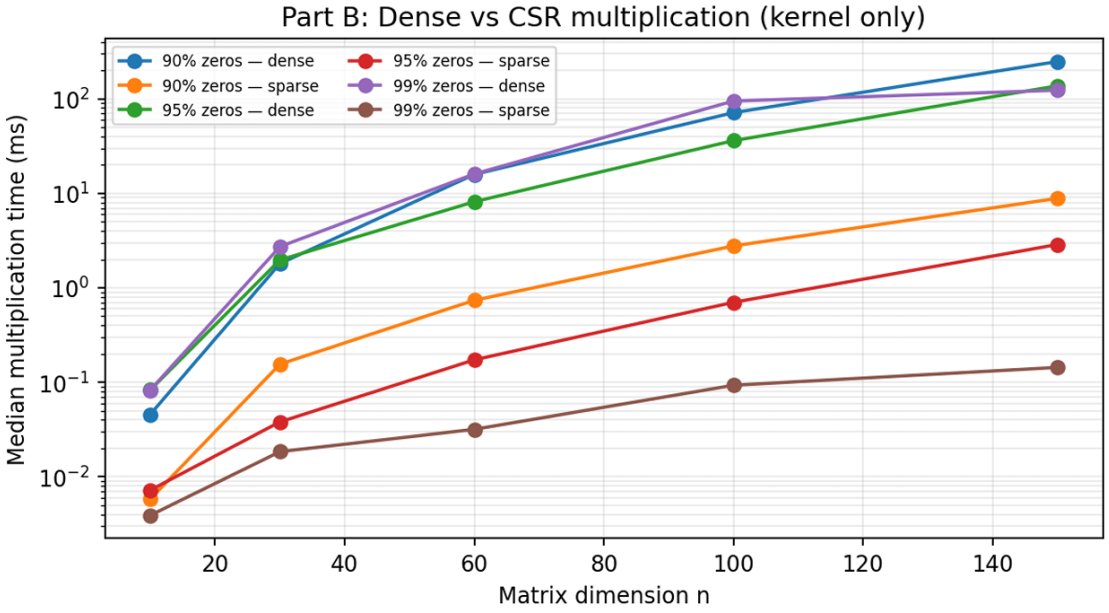
3.2. Part A memory estimate.

The CSV reports a matrix payload of 2,400 bytes at n = 10, 21,600 bytes at n = 30, 86,400 bytes at n = 60, 240,000 bytes at n = 100, and 540,000 bytes at n = 150. These values correspond to the reported payload accounting for the three n × n matrices (A, B, and output C) at eight bytes per element. They do not include Python list/object overhead, temporary objects, interpreter memory, or peak process RAM.

4. PART B – SPARSE MULTIPLICATION.

Part B compares dense multiplication against the CSR-based sparse method at target zero proportions of 90%, 95%, and 99%. The main timing comparison below is multiplication-kernel time only; conversion is considered separately afterwards.

n	Zeros	Dense Median (ms)	CSR median (ms)	CSR payload (B)	Dense payload (B)
10	90%	0.046	0.006	632	2400
10	95%	0.083	0.007	600	2400
10	99%	0.083	0.004	296	2400
30	90%	1.804	0.155	8072	21600
30	95%	1.971	0.038	3080	21600
30	99%	2.725	0.018	952	21600
60	90%	15.857	0.738	41176	86400
60	95%	8.148	0.173	16680	86400
60	99%	16.041	0.032	3000	86400
100	90%	71.505	2.774	133176	240000
100	95%	36.157	0.702	54232	240000
100	99%	94.909	0.093	7640	240000
150	90%	248.185	8.849	352760	540000
150	95%	137.551	2.870	149016	540000
150	99%	123.527	0.144	16728	540000



4.1. Multiplication Performance.

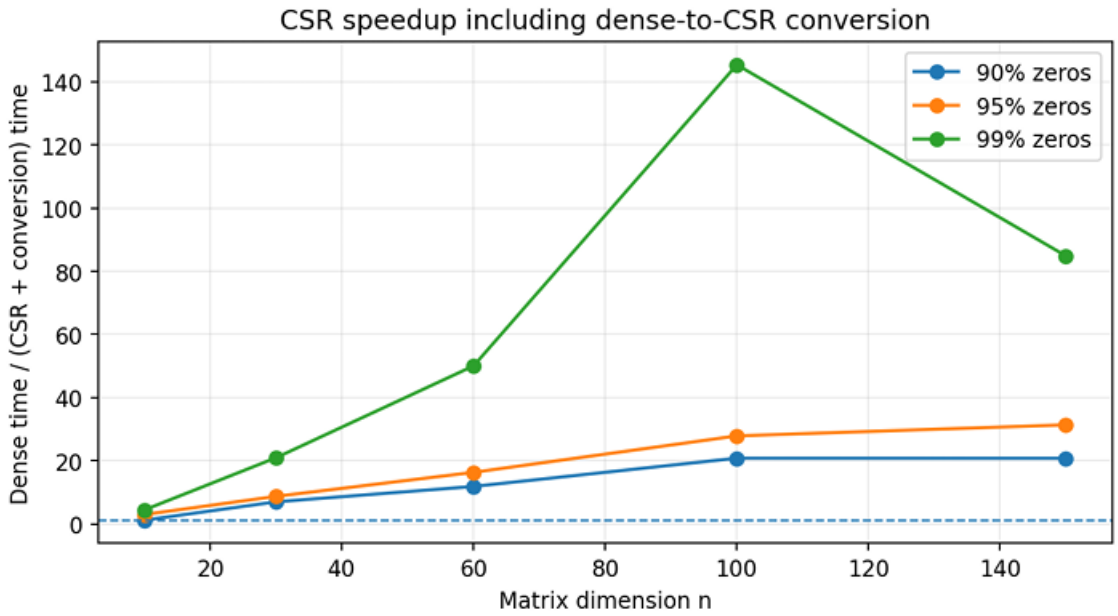
CSR has a median multiplication time than dense multiplication in all 15 tested configurations. The measured advantage is especially large at sparsity, where fewer non-zero entries need to be processed. For example at n = 150 and 99 percent target zeros the dense median is 123.527 ms while CSR takes 0.144 ms for multiplication alone. At n = 150 and 90 percent target zeros the respective medians are 248.185 ms and 8.849 ms.

The size and sparsity patterns are not perfectly monotonic in the measurements. For example at n = 60 the dense median for 95 percent target zeros is lower than the medians for 90 percent and 99 percent. Dense multiplication does not exploit the zero pattern so these differences are plausibly associated with measurement runtime variability than, with the target sparsity itself. They should not be interpreted as evidence that more zeros make dense multiplication slower or faster.

4.2. Conversion-aware Comparison

To estimate the cost when inputsre, in dense format the measured conversion time was added to the median CSR multiplication time: $T_{CSR,total} = T_{conversion} + median(T_{CSR\ multiplication})$. The conversion value was recorded once for each size/sparsity configuration. Reused across the matching CSV rows. It is therefore considered a configuration-level measurement, not a repeated timing distribution.

n	Zeros	Dense Median (ms)	CSR median (ms)	CSR payload (B)	Dense payload (B)	Speedup incl. conversion
10	90%	0.046	0.006	632	2400	1.20×
10	95%	0.083	0.007	600	2400	2.98×
10	99%	0.083	0.004	296	2400	4.44×
30	90%	1.804	0.155	8072	21600	6.99×
30	95%	1.971	0.038	3080	21600	8.73×
30	99%	2.725	0.018	952	21600	20.96×
60	90%	15.857	0.738	41176	86400	11.86×
60	95%	8.148	0.173	16680	86400	16.33×
60	99%	16.041	0.032	3000	86400	50.03×
100	90%	71.505	2.774	133176	240000	20.81×
100	95%	36.157	0.702	54232	240000	27.86×
100	99%	94.909	0.093	7640	240000	145.34×
150	90%	248.185	8.849	352760	540000	20.79×
150	95%	137.551	2.870	149016	540000	31.35×
150	99%	123.527	0.144	16728	540000	84.85×



CSR stays faster even after conversion is added in all 15 configurations. When the size is 150 I find that the estimated total time for CSR is 11.938 milliseconds for a matrix that's 90% zero 4.387 milliseconds for 95% zero and 1.456 milliseconds for 99% zero. By contrast the median times for representation are 248.185 milliseconds, 137.551 milliseconds and 123.527 milliseconds. This estimate is aware of the conversion cost. It does not cover the whole end-to-end process. It does not include costs like repeated conversions, memory allocation or turning the result back into a dense format, for checking.

4.3. Storage payload.

For the 150 configurations that were reported the estimated dense payload comes to 540,000 bytes. The benchmark shows the CSR payload as 352,760 bytes at 90% target zeros 149,016 bytes at 95% and 16,728 bytes at 99%. This shows a big drop in payload as sparsity goes up for the matrices being tested.. The CSR estimate only includes values, column indices and row pointers. It leaves out Python object. So it’s not the same, as memory use or peak RAM usage.

4.4. Non-zero Counts and Output Structure.

The sparse CSV lists non-zero counts for both the inputs and the output. At n = 150 the input counts (A B) and the output non-zero count are: (2,192, 2,276, 17,353) when 90% of the entries are zeros; (1,158, 1,079 6,850) when 95% of the entries are zeros; and (221, 252 346) when 99% of the entries are zeros.

The output can hold non-zero elements than either of the inputs because multiplying and adding can create non-zero output entries in spots that were zero, in the inputs. So input sparsity alone does not fully decide output sparsity or the cost of CSR.

5. DISCUSSION.

The dense results indicate that low-level implementation choices improve performance for specific sizes, though they are not universally beneficial across all dimensions. Specifically, while low-level optimizations prove useful for certain sizes, they do not scale effectively to every size.

The blocked implementation achieves the median performance when $n = 30$, whereas the reordered implementation yields the optimal medians at $n = 100$ and $n = 150$. The experiment does not identify the underlying causes for performance fluctuations at specific sizes; further code-level analysis across a broader range of sizes is required to determine the impact of cache and interpreter effects.

In contrast, the sparse results strongly favor the Compressed Sparse Row (CSR) format for the tested matrices and sizes, even when accounting for the one-time conversion cost. This advantage becomes more pronounced as sparsity increases. However, these conclusions remain constrained by the current implementation and workload. CSR may introduce indexing and conversion overhead, and its performance benefits could diminish with lower sparsity or when the output is relatively dense. Because the experiment did not evaluate break-points or larger matrices, these observations should be interpreted as general insights rather than definitive measured results.

6. LIMITATIONS.

Several limitations exist: five matrix sizes up to 150 × 150 were tested each configuration has five timed repetitions conversión time was measured once, per configuration memory is estimated by payload instead of measured peak RAM sparse multiplication timing excludes the CSR-output to dense conversión used for validation and runtime variability shows up in some dense measurements.

The results should not be generalized beyond the tested conditions without experiments.

7. CONCLUSION.

Part A shows that no dense implementation was the fastest for every matrix size. The baseline dense implementation was fastest when n equals 10. The blocked dense implementation was fastest when n equals 30. The reordered dense implementation had the median time when n equals 100 and when n equals 150. At n equals 60 the dense implementations were close. This indicates that the effectiveness of an optimisation depends on the tested input size and on the implementation chosen.

Part B shows that the CSR format had a median multiplication time than dense multiplication for every tested size and sparsity configuration. After adding the separately measured conversion time the CSR format remained faster in all fifteen configurations and the strongest advantage was measured at a target of ninety-nine percent zeros. The payload estimates also fell as the target sparsity increased. These results support the use of CSR, for the tested workload but the conclusions of the report are still limited by the tested sizes, the conversion methodology and the payload-only memory estimates.